STUDENT EXPERIMENT GUIDE

Purpose: This combined guide explains how students can complete each experiment and which free or open-source tools they can use. The activities are designed to avoid paid subscriptions and paid cloud deployments.

Important cost and access note: Some online services have account requirements, usage caps, or time-limited trials; these are optional and must not be required for completion. A free trial is not the same as freeware. Microsoft Power Apps, Power Automate, Dataverse, Power Pages, Copilot Studio, Microsoft Purview, and Azure are proprietary services. Where these are named in a question, students can submit a free-tool prototype or a clearly labelled design unless the institution provides an authorized licence.

General method for every experiment

- Read the scenario and identify the problem, inputs, expected outputs, and constraints.
- Plan the data fields, tools, workflow steps, rules, and security needs before building.
- Use synthetic or fictional data unless real data use has been explicitly authorized.
- Build a small prototype with free tools where practical; otherwise provide a diagram or structured design.
- Test normal cases, boundary cases, invalid inputs, and access permissions where relevant.
- Compare expected and actual results, and explain errors or limitations.
- Submit the objective, design, data/sample inputs, screenshots or diagrams, test cases, results, and conclusion.

Recommended free/open-source toolset

- Python, JupyterLab, pandas, and scikit-learn — programming, data preparation, machine learning, and evaluation
- SQLite — local database storage
- LibreOffice — documents, spreadsheets, and tables
- diagrams.net (draw.io) — architecture and workflow diagrams
- Tesseract OCR — text extraction from scanned forms
- Whisper / whisper.cpp — speech-to-text
- OpenCV — image processing

- Ollama — local generative AI, if computer hardware supports it
- Visual Studio Code — free code editor
- OpenSSL — cryptography demonstrations

1. AI Fundamentals and Applications

How to complete the experiment:

- Choose four engineering examples: one each for AI, machine learning, generative AI, and an AI agent.
- Create a comparison table with columns: concept, problem, input, processing, output, and example.
- Use a simple example such as sensor-based fault prediction for machine learning, a text explanation for generative AI, and a tool-using assistant for an AI agent.
- Explain why each technology is suitable for its example. A working application is not essential unless the examiner asks for one.
- Submit: comparison table and a short explanation of each example.

Free tools and practical notes:

- LibreOffice Calc or Python for a comparison table and simple examples.
- Python with scikit-learn for a small machine-learning example.
- Ollama with a locally downloaded open model, if the computer has sufficient resources, for a generative-AI demonstration.
- Use a simple Python script with fixed rules/tools to demonstrate an agent-like workflow.
- All can be used without paid subscriptions. Downloading and running local models may require a capable computer.

2. NLP and Information Extraction

How to complete the experiment:

- Prepare 10–20 sample feedback comments and 3–5 sample forms. Use fictional data; do not upload real student personal information to unapproved AI tools.
- Use an AI assistant to summarize the comments and group common themes such as teaching, facilities, or course content.
- Use an OCR or document-extraction tool available to you to read the sample forms and extract selected fields.
- Put extracted fields into a table with one row per form.

- Compare every extracted field with the original form, mark errors, and correct them.
- Submit: sample inputs, summary, extracted-data table, validation results, and a short privacy note.

Free tools and practical notes:

- LibreOffice Writer/Calc for sample feedback and the output table.
- Tesseract OCR (free and open source) for extracting text from scanned sample forms.
- Python with pandas and basic text-processing libraries for grouping or counting themes.
- Optional: Ollama with a local model for summarization, subject to local hardware capacity.
- Use fictional sample forms. A local model is optional; a rules-based summary or manual comparison can be used if the computer cannot run one.

3. Speech and Computer Vision

How to complete the experiment:

- Prepare 3–5 spoken questions about a lab component and record or speak them into an available speech-to-text tool.
- Check whether the recognized text matches what was spoken; note any errors caused by noise, accent, or technical words.
- Use an available text-to-speech tool to read a sample answer aloud.
- Take clear photos of a component, including one normal example and, if safe and available, one example with a visible defect.
- Use an approved vision-capable AI tool or a simple image-classification tool to describe visible features. Do not treat the result as a certified safety inspection.
- Compare the output with what is actually visible and record limitations.
- Submit: audio/transcription evidence, spoken output, images, model responses, and observations.

Free tools and practical notes:

- Whisper or whisper.cpp for local speech-to-text.
- An offline text-to-speech tool such as Piper, where compatible with the student's system.
- OpenCV and Python for image handling or a simple computer-vision demonstration.
- Use student-taken photos of safe, ordinary components; verify all AI descriptions manually.
- These are free/open-source software options. Setup and model downloads may require storage and computing resources.

4. Responsible AI

How to complete the experiment:

- Use the job-shortlisting scenario in the question and list possible risks under fairness, reliability, privacy, inclusion, transparency, and accountability.
- For each risk, write a realistic example of how harm could occur.
- Add a mitigation beside each risk—for example, test results across groups, human review, minimum data collection, accessible processes, documented reasons, and an appeal route.
- Explain which decisions should remain subject to human review.
- Submit: risk-and-mitigation table and a short conclusion.

Free tools and practical notes:

- LibreOffice Writer or Calc to prepare a risk-and-mitigation table.
- Use a fictional hiring scenario and assess fairness, reliability, privacy, inclusion, transparency, and accountability.
- No paid AI platform is required.
- Entirely possible with free desktop software.

5. Machine Learning Problem Formulation

How to complete the experiment:

- Create or obtain a small, labelled sample dataset of machine sensor readings. Include a target such as failure within the next 24 hours (yes/no).
- Choose features such as temperature, vibration, pressure, operating hours, and error count.
- Check missing values, duplicate rows, inconsistent units, and whether the target is correctly labelled.
- Separate training and test data before evaluating the model. If time-ordered sensor data is used, avoid randomly mixing future records into the training set.
- Train a simple classification model using a tool available in the lab, or provide a clear model-design plan if software access is unavailable.
- Evaluate with a confusion matrix, precision, recall, and F1-score; explain why missing a real failure may be costly.
- Submit: dataset description, target/features, split method, evaluation results, and limitations.

Free tools and practical notes:

- Python and JupyterLab, or Google Colab's free tier if online execution is acceptable.
- Use pandas to clean a synthetic CSV dataset and scikit-learn to train a basic classifier.
- Use scikit-learn metrics such as confusion matrix, precision, recall, and F1-score.
- Use synthetic data if real factory data is unavailable.
- Local Python/Jupyter is free. Hosted notebook quotas and availability may change; no paid upgrade is required for the basic exercise.

6. Model Evaluation

How to complete the experiment:

- Prepare three small example problems: predict energy usage, classify a product as defective/non-defective, and group machines by similar behaviour.
- Label each task as regression, classification, or clustering and explain why.
- For regression, choose metrics such as MAE or RMSE and compare predictions with actual values.
- For classification, create a confusion matrix and report precision, recall, and F1-score as appropriate.
- For clustering, inspect whether groups are meaningful; use a metric such as silhouette score only when appropriate and explain its limits.
- Explain which error matters most for the business use case.
- Submit: comparison table, example metrics, and interpretation.

Free tools and practical notes:

- Python, pandas, and scikit-learn for regression, classification, and clustering examples.
- Use MAE or RMSE for regression, precision/recall/F1 for classification, and a suitable clustering evaluation such as silhouette score when applicable.
- Use LibreOffice Calc if a table-based explanation is sufficient.
- Free desktop tools are sufficient.

7. Power Platform Component Selection

How to complete the experiment:

- Write the requirements as separate functions: student registration, attendance alerts, data storage, and reporting.
- Map each requirement to a component: Power Apps for the user interface, Dataverse or an approved data source for records, Power Automate for alerts, and Power BI for dashboards.
- Draw a simple flow diagram showing where information is entered, stored, processed, and displayed.
- If access is available, create a small prototype or sample screen; otherwise present the design and explain the integration.
- Submit: component-selection table, flow diagram, and screenshots if a prototype is built.

Free tools and practical notes:

- Use diagrams.net (draw.io) to map the requirements to app, data, workflow, and dashboard components.
- Use LibreOffice Calc to create a component-selection table and sample dashboard data.
- If the question specifically requires Microsoft Power Apps, Power Automate, Dataverse, or Power BI, explain that these are proprietary products and that a free trial/developer plan is not the same as freeware.
- For a strict no-cost/no-trial requirement, submit a design using free alternatives rather than depending on a Microsoft subscription.

8. Power Apps and Dataverse

How to complete the experiment:

- Define an Equipment table with fields such as EquipmentID, Name, Location, Condition, Status, and LastUpdated.
- Create a canvas app with a list/browse screen, a detail screen, and a form to add or edit records.
- Add validation: EquipmentID must be present and unique; status must be selected from allowed values; required fields cannot be blank.
- Connect the app to Dataverse or the data source approved for the course.
- Configure roles so ordinary users can view records while authorized staff can update them, according to the scenario.

- Test adding a valid record, submitting an invalid record, searching for equipment, and trying an action without permission.
- Submit: data schema, app screenshots, validation rules, permission design, and test results.

Free tools and practical notes:

- Use Python with Streamlit or Flask to build a simple equipment-management interface.
- Use SQLite as the local database and define EquipmentID, Name, Location, Condition, Status, and LastUpdated fields.
- Implement required-field checks, allowed status values, and role checks in the prototype.
- Use diagrams.net for the data model if coding is not required.
- Python, Streamlit/Flask, and SQLite are free. This demonstrates the concepts but is not a Power Apps/Dataverse implementation.

9. Power Automate Workflow

How to complete the experiment:

- Prepare sample attendance records with student ID, email or mentor contact, and attendance percentage.
- Choose and document a threshold, such as 75%, as an example assumption unless the institution specifies another value.
- Create a trigger, such as a scheduled recurrence or a new/updated attendance record.
- Read the relevant record and add a condition comparing attendance with the threshold.
- If attendance is below the threshold, send a test notification to a designated test recipient; otherwise take no alert action or record that no alert is needed.
- Test values below, equal to, and above the threshold, and confirm that the flow behaves as intended.
- Submit: flow diagram or screenshots, condition logic, test cases, and results.

Free tools and practical notes:

- Use Python to read a sample CSV attendance file and compare each student's percentage with a stated threshold.
- Use a scheduled local script or a self-hosted n8n Community Edition workflow if a visual workflow is desired.

- Print or save test notifications instead of sending messages to real students.
- Test below-threshold, equal-to-threshold, and above-threshold values.
- Python is free; self-hosting n8n may require setup. Do not assume third-party email or SMS delivery is free.

10. Power Pages

How to complete the experiment:

- Sketch a student portal with application submission, confirmation, and status-check pages.
- Define the application fields and the statuses, such as Submitted, Under Review, Approved, or Rejected.
- Specify where records will be stored and how the authenticated student will be linked to their own application records.
- Design permissions so students cannot see other students' applications; define staff access separately.
- Describe or configure a notification workflow for submission and status changes.
- Test with two sample student accounts, verifying that each account can access only its permitted records.
- Submit: page sketches or screenshots, data model, permissions matrix, workflow, and test results.

Free tools and practical notes:

- Use Python with Flask or Django to design a small student application portal.
- Use SQLite for application records and create submission and status pages.
- Demonstrate authentication/authorization concepts and ensure each test user can access only their own sample record.
- Use a local simulated notification log rather than paid email/SMS services.
- Free/open-source tools can demonstrate the concepts, but the result is not a Microsoft Power Pages implementation.

11. Prompt Engineering

How to complete the experiment:

- Start with the vague request: “Summarize the quarterly numbers.”
- Write the Context: identify the team, reporting period, and business situation.
- Write the Goal: specify what decision or insight the summary should support.
- Write the Source: name the supplied report or dataset and instruct the AI not to invent missing values.
- Write Expectations: specify length, headings, comparisons, important metrics, and output format.
- Run the prompt on a sample dataset or document.
- Check every number and conclusion against the source; record any omission or unsupported claim.
- Submit: original prompt, improved prompt, AI output, and validation notes.

Free tools and practical notes:

- Use a locally running model through Ollama if available, or any institution-approved free AI access with clearly stated limits.
- Create a prompt with Context, Goal, Source, and Expectations.
- Use a fictional quarterly report and validate every figure in LibreOffice Calc.
- Do not upload confidential reports to public AI services.
- A local model avoids a paid API requirement, though local hardware is needed. Free hosted access may have usage limits.

12. Grounding and Declarative Agents

How to complete the experiment:

- Collect a small set of approved, non-confidential academic policy or handbook documents.
- Define the agent's role and instructions—for example, answer only from the approved sources and state when evidence is missing.
- Configure or sketch the knowledge sources and explain how the agent should retrieve relevant passages.
- Define access rules for documents that different users are permitted to view.
- Ask test questions that are answered by the documents and questions that are not covered.
- Check whether answers match the source and whether unsupported questions receive a clear limitation or referral.
- Submit: agent instructions, source list, sample questions and answers, access design, and evaluation table.

Free tools and practical notes:

- Use Ollama with a local model and a small set of public or fictional handbook documents.
- Use a simple local retrieval prototype with Python; optionally use a free/open-source framework such as LlamaIndex or LangChain.
- Ask answerable and unanswerable questions and check whether the response is supported by the source text.
- Demonstrate access-control design with separate sample roles; do not use confidential documents.
- Free/open-source components are available, but setup can be technical. This is not a Microsoft Copilot Studio declarative agent.

13. Agent vs. Automation Workflow

How to complete the experiment:

- Create a sample milestone table containing project, milestone, due date, owner, and status.
- Define the deadline rule precisely—for example, alert when the due date has passed and status is not Complete.
- Choose a scheduled or event-based trigger and describe how records will be checked.
- Use deterministic workflow logic to identify missed deadlines and send a notification to the responsible person.
- Explain whether an AI agent adds value, such as summarizing project notes, while leaving deadline calculations to fixed rules.
- Test a completed milestone, a future deadline, and an overdue incomplete milestone.
- Submit: architecture diagram, rule logic, test cases, and justification for using automation and/or an agent.

Free tools and practical notes:

- Store sample milestones in CSV or SQLite.
- Use Python or a self-hosted n8n Community Edition workflow to check whether due dates have passed and status is incomplete.
- Log notifications to a file or console instead of relying on paid messaging services.
- Test completed, future-due, and overdue-incomplete milestones.
- No paid cloud services are required for a local demonstration.

14. GitHub Copilot and Code Review

How to complete the experiment:

- Write a clear prompt asking for a function that calculates average order value from valid order amounts and specify the programming language.
- Define how an empty list, missing values, negative amounts, and non-numeric inputs should be handled.
- Review the generated code for correctness, readability, security, edge cases, and alignment with the prompt.
- Run test cases such as [100, 200] → 150; an empty list → the documented empty-input result; and invalid input → the documented error or handling behaviour.
- Run the tests and revise the code if any result is wrong.
- Explain why the generated code must be reviewed and tested before deployment.
- Submit: prompt, generated code, test cases, results, and review notes.

Free tools and practical notes:

- Use Visual Studio Code or another free code editor.
- Write the average-order-value function in Python and test empty, missing, negative, and non-numeric values.
- Use Python's built-in unittest or pytest to run tests.
- An AI coding assistant is optional. If using one, choose a free tier only after checking its current limits; the coding and testing task can be completed without AI assistance.
- The core experiment requires no paid AI assistant.

15. Deterministic Agent Flow

How to complete the experiment:

- Define the input fields: employee ID, leave start date, and number of leave days.
- Validate that the employee exists, the date is valid, and the number of days is a positive permitted value.
- Retrieve the employee's leave balance and calculate whether enough days are available.
- Apply the stated approval rules and route the request to the appropriate approver when necessary.
- Create clear outcomes for invalid requests, insufficient balance, pending approval, approval, and rejection.
- Record request ID, employee ID, timestamps, checks, decision, approver, and status changes for audit purposes.

- Test valid, invalid, insufficient-balance, approved, and rejected scenarios.
- Submit: flowchart, rules, sample inputs/outputs, and audit-field list.

Free tools and practical notes:

- Use Python to validate employee ID, date, leave days, and leave balance against fictional records in CSV or SQLite.
- Apply fixed approval rules and return clear outcomes for valid, invalid, insufficient-balance, approved, and rejected requests.
- Write an audit log to a local file or database.
- Test each outcome using fictional employee data.
- Python and SQLite are free; no HR platform subscription is required.

16. Azure AI Architecture

How to complete the experiment:

- Draw the flow from user to application, API, AI model/agent, knowledge retrieval, and response.
- Identify where approved documents or other knowledge sources are stored.
- Show how identity and access management determines which users and documents can be accessed.
- Identify where secrets and credentials are stored securely rather than hard-coded in code.
- Describe network protection, hosting, scaling, and logging/monitoring.
- Explain how the system retrieves relevant content, generates an answer, and prevents unauthorized disclosure.
- Submit: labelled architecture diagram, component descriptions, and a security checklist.

Free tools and practical notes:

- Use diagrams.net to draw the application, API, model, knowledge store, identity, secrets, network, hosting, and monitoring layers.
- Use local open-source components for a prototype, such as Python, Ollama, and SQLite, if a working demo is required.
- Describe Azure services conceptually if the question explicitly asks for Azure architecture, but do not deploy paid cloud resources.
- Clearly label the result as a design if no Azure deployment was performed.
- Architecture diagrams are free. Azure free offers and trials have terms/limits and are not the same as permanently free software.

17. Cryptography and Zero Trust

How to complete the experiment:

- Create a table explaining symmetric encryption, asymmetric encryption, hashing, and digital signatures, including purpose and a typical use.
- Use a simple example to distinguish encryption, which protects confidentiality, from hashing, which helps verify integrity.
- For the contractor scenario, specify identity verification and multi-factor authentication where required.
- Explain how device condition, location, risk, and business need affect access decisions.
- Apply least privilege and time-limited access where appropriate.
- Describe monitoring and conditions that should cause access to be denied or revoked.
- Submit: comparison table and a Zero Trust decision flow.

Free tools and practical notes:

- Use Python's standard library for hashing demonstrations; use the free/open-source OpenSSL command-line tool for suitable encryption and digital-signature demonstrations.
- Use fictional data only and explain the difference between encryption, hashing, and signatures.
- Draw a Zero Trust access decision flow in diagrams.net.
- Do not test access against real databases or systems without written authorization.
- Free local tools are sufficient for a classroom demonstration.

18. Microsoft Purview and Data Protection

How to complete the experiment:

- Identify sample sensitive data such as fictional employee IDs and product codes.
- Describe how the organization could discover or classify those data types.
- Propose suitable sensitivity labels and explain which users should be allowed to access the information.
- Define a data loss prevention rule that detects or blocks an unauthorized sharing action.
- Explain retention and monitoring requirements and who should review alerts.
- Test the policy using fictional sample content in an approved environment, if available.
- Submit: data-classification table, proposed policy, expected action, and test evidence or design notes.

Free tools and practical notes:

- Use LibreOffice Calc to create fictional employee IDs and product codes and classify them.
- Draft example sensitivity labels, a data-loss-prevention rule, retention rules, and monitoring steps in a document.
- Use a local Python script to flag sample identifiers or prohibited sharing patterns if a simple demonstration is needed.
- State that this is a policy simulation, not an actual Microsoft Purview configuration.
- Microsoft Purview is a proprietary service. Its live configuration should not be required when the requirement is strict freeware only.

19. Cybersecurity Threat Analysis

How to complete the experiment:

- Write a brief incident timeline from the discovery of malware and the reported phishing click.
- List possible indicators to investigate, such as suspicious processes, network connections, email links, and authentication logs.
- Describe containment actions, including isolating affected devices according to the organization's incident process.
- Explain how to investigate the initial access, determine the scope, remove malicious activity, and restore systems safely.
- State when credentials or sessions should be revoked and passwords reset, following security-team guidance.
- Describe how to warn users, report the phishing message, and prevent recurrence.
- Submit: incident-response checklist, timeline, evidence list, and recovery criteria.

Free tools and practical notes:

- Use a fictional incident scenario and prepare a timeline, indicators, containment plan, recovery checklist, and user guidance in LibreOffice.
- Optionally analyse safe sample logs with Python.
- Use only an authorized training lab; never download or execute real malware for this experiment.
- Do not conduct phishing tests against real users without written authorization.

- A written tabletop exercise can be completed entirely with free tools.

20. Integrated Capstone Project

How to complete the experiment:

- Choose a manufacturing scenario and define the equipment, products, users, and business problem.
- Design a sensor-data model and choose an ML task for predicting failure; define target, features, and evaluation metrics.
- Design an image-analysis workflow for identifying visible product defects and explain how labelled images would be evaluated.
- Design a deterministic maintenance-alert workflow with thresholds, recipients, and escalation rules.
- Create or design an assistant grounded in approved equipment manuals; explain how it handles unsupported questions.
- Draw the complete architecture and data flow, including application, data storage, AI components, workflows, and monitoring.
- Specify access controls, privacy safeguards, human review, and limitations.
- Prepare at least five test cases with inputs, expected outputs, and actual results where a prototype is available.
- Submit: problem statement, architecture, data schema, AI/model plan, workflow, security controls, tests, and a demonstration or design presentation.

Free tools and practical notes:

- Use synthetic sensor data in CSV and Python/scikit-learn to demonstrate failure prediction.
- Use OpenCV and, if appropriate, a locally available open model to inspect sample product images; describe limitations and verify outputs.
- Use Python rules or self-hosted n8n to generate simulated maintenance alerts.
- Use Ollama with local public/fictional equipment manuals for a basic document-grounded assistant, or submit a retrieval architecture if local hardware is insufficient.
- Draw the complete architecture with diagrams.net and test at least five scenarios.
- Document security, privacy, human review, metrics, limitations, and expected versus actual results.

- This can be completed with free/open-source tools and synthetic data. Some local AI models need substantial memory or processing power; a design-only alternative is acceptable where hardware is limited.

Final student submission checklist

- Experiment title and objective
- Tool names and confirmation that no paid feature was required
- Sample or synthetic input data
- Steps followed and/or architecture or workflow diagram
- Code/configuration, screenshots, or sample outputs where available
- At least three test cases for small workflows and five for the capstone
- Expected result versus actual result
- Security, privacy, and Responsible AI considerations
- Limitations and conclusion